

STAT 408 Lab 3: tidyr

Hadley Stabnau

2026-01-29

```
library(tidyverse)
library(dplyr)
library(nycflights13)
```

Summary statistics

Exercise 1

Write code to compute the average departure delay for each carrier. Remember to give your new variable a sensible name. Make sure to ignore missing data when computing your summary statistics.

```
flights |>
  summarize(avg_dep_delay=mean(dep_delay, na.rm=TRUE))
```

```
# A tibble: 1 x 1
  avg_dep_delay
      <dbl>
1          12.6
```

Exercise 2

Write code to compute the shortest, longest, and average time a flight spent in the air, by month. Remember to give your new variable a sensible name. Make sure to ignore missing data when computing your summary statistics.

```
flights |>
  group_by(month) |>
  summarize(short_air_time=min(air_time, na.rm=TRUE))
```

```
# A tibble: 12 x 2
  month short_air_time
  <int>         <dbl>
1     1           20
2     2           21
3     3           21
4     4           20
5     5           21
6     6           21
7     7           23
8     8           21
9     9           21
10    10           23
11    11           24
12    12           21
```

```
flights |>
  group_by(month) |>
  summarize(long_air_time=max(air_time, na.rm = TRUE))
```

```
# A tibble: 12 x 2
  month long_air_time
  <int>         <dbl>
1     1          667
2     2          691
3     3          695
4     4          671
5     5          640
6     6          650
7     7          629
8     8          640
9     9          636
10    10          642
11    11          676
12    12          661
```

```
flights |>
  group_by(month) |>
  summarize(avg_air_time=mean(air_time, na.rm=TRUE))
```

```
# A tibble: 12 x 2
  month avg_air_time
  <int>         <dbl>
1     1          154.
2     2          151.
3     3          149.
```

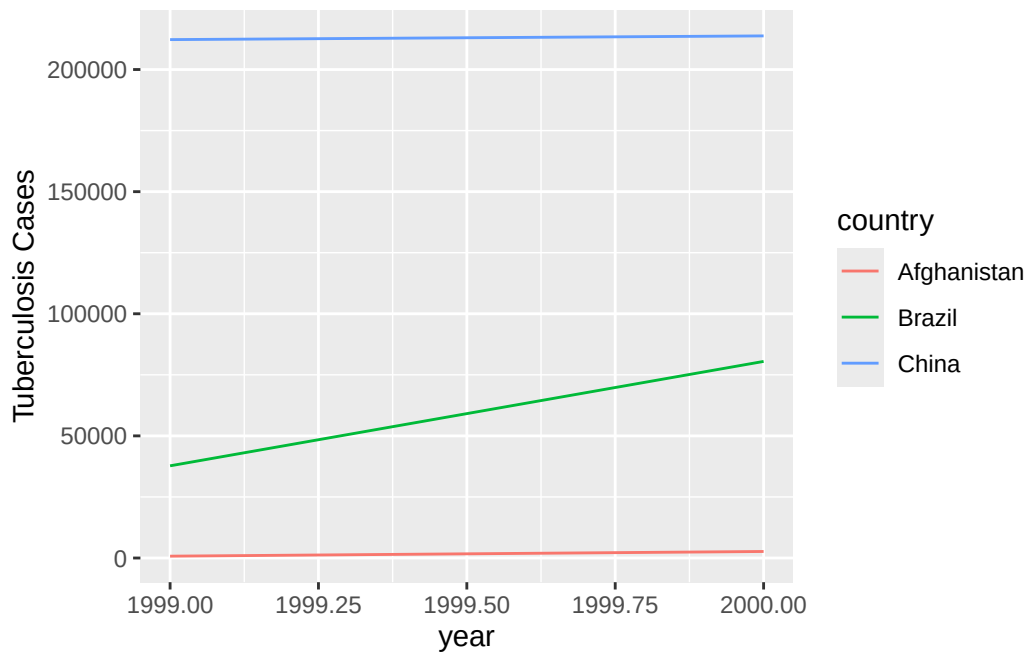
4	4	153.
5	5	146.
6	6	150.
7	7	147.
8	8	148.
9	9	143.
10	10	149.
11	11	155.
12	12	163.

Tidying data

Exercise 3

Using the `tidyr` dataset `table1`, make a `ggplot` plot that shows a (very short) time series for each country of the number of tuberculosis cases. In other words, the horizontal axis should be time, the vertical axis should be cases, and you should have one line per country.

```
library(tidyr)
table1 |>
  ggplot(aes(x=year, y=cases, group=country, color=country)) +
  geom_line() +
  labs(x="year", y="Tuberculosis Cases")
```



Exercise 4

Make some notes for yourself:

How can you tell whether you need `pivot_longer` or `pivot_wider`?

- I can tell when I need `pivot_longer` vs `pivot_wider` based on how I want my data to look. If I want the data to look like fewer columns and more rows, I'd use `pivot_longer`. On the flip side, if I wanted my data to show a lot of columns and only the few rows, I'd use `pivot_wider`. It seems that at least with one-sided tables, `pivot_longer` puts variables across the top and `pivot_wider` puts the variables down the side.

How can you remember what basic information you need to give these functions and what those arguments are called?

- Some basic information needed to make these functions work are arguments like `min`, `max` and `mean` to find averages and smallest/largest values in a set. It's also good to remember that `summarize()` gives back a single value as opposed to ranking data by `arrange()`.

How can you figure out the answers to the previous question when you don't remember?

- If I didn't remember, I could either look back at the lab03 notes or just back at my code above that was successful and see what arguments were used in order to make the functions run correctly. I could also go back to my reference sheet from HW 2.

Exercise 5

Let's check out a `tidyr` dataset called `relig_income`. Remember, you don't have to do anything to load or use the dataset as long as `tidyr` is loaded, though if you want to see it in the Environment pane so you can click on it and visualize it, you can use `data(relig_income)`.

This dataset tells you some survey results on respondents' religion and income level. Specifically, there's a row for each religion, a column for each income level, and the value of the cell is how many respondents reported that religion and income level.

Use `names(relig_income)` to see what the variables of this dataset are. Then pivot the dataset so that there is a `religion` column, a column called `income` indicating the income level (such as `<$10k`), and a `count` column that tells you how many people had that combination of religion and income level.

```
data(relig_income)
names(relig_income)
```

```
relig_income1<- relig_income |>
  pivot_longer(cols=-religion, names_to= "income", values_to= "count")

relig_income1
```

```
# A tibble: 180 x 3
  religion income      count
  <chr>    <chr>    <dbl>
1 Agnostic <$10k      27
2 Agnostic $10-20k    34
3 Agnostic $20-30k    60
4 Agnostic $30-40k    81
5 Agnostic $40-50k    76
6 Agnostic $50-75k   137
7 Agnostic $75-100k  122
8 Agnostic $100-150k 109
9 Agnostic >150k     84
10 Agnostic Don't know/refused 96
# i 170 more rows
```

Exercise 6

Pick two examples from the [pivoting vignette in the R documentation](#), one example of `pivot_longer` and one of `pivot_wider`, that you think are useful for your own data and applications. Explain them here in your own tutorial, using a mix of text, code, and code comments. In other words, write a tutorial like the pivoting vignette linked above to guide someone through these examples. Help your reader (i.e. your future self!) understand the original structure of the dataset, what dataset you want to transform it into using pivoting, and how to use pivoting to get there (including what arguments you need and why).

```
data(billboard)
names(billboard)
```

```
[1] "artist"      "track"      "date.entered" "wk1"        "wk2"
[6] "wk3"         "wk4"         "wk5"         "wk6"         "wk7"
[11] "wk8"         "wk9"         "wk10"        "wk11"        "wk12"
[16] "wk13"        "wk14"        "wk15"        "wk16"        "wk17"
[21] "wk18"        "wk19"        "wk20"        "wk21"        "wk22"
[26] "wk23"        "wk24"        "wk25"        "wk26"        "wk27"
[31] "wk28"        "wk29"        "wk30"        "wk31"        "wk32"
[36] "wk33"        "wk34"        "wk35"        "wk36"        "wk37"
[41] "wk38"        "wk39"        "wk40"        "wk41"        "wk42"
[46] "wk43"        "wk44"        "wk45"        "wk46"        "wk47"
[51] "wk48"        "wk49"        "wk50"        "wk51"        "wk52"
[56] "wk53"        "wk54"        "wk55"        "wk56"        "wk57"
```

[61]	"wk58"	"wk59"	"wk60"	"wk61"	"wk62"
[66]	"wk63"	"wk64"	"wk65"	"wk66"	"wk67"
[71]	"wk68"	"wk69"	"wk70"	"wk71"	"wk72"
[76]	"wk73"	"wk74"	"wk75"	"wk76"	

```
billboard1<-billboard |>
  pivot_longer(cols= starts_with("wk"), names_to = "week", values_to = "rank", values_drop_na = TRUE)

billboard1
```

A tibble: 5,307 x 5

	artist	track	date.entered	week	rank
	<chr>	<chr>	<date>	<chr>	<dbl>
1	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk1	87
2	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk2	82
3	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk3	72
4	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk4	77
5	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk5	87
6	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk6	94
7	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk7	99
8	2Ge+her	The Hardest Part Of ...	2000-09-02	wk1	91
9	2Ge+her	The Hardest Part Of ...	2000-09-02	wk2	87
10	2Ge+her	The Hardest Part Of ...	2000-09-02	wk3	92

i 5,297 more rows

- This dataset originally had columns labeled: artist, track, date.entered, wk1, wk2, wk3 all the way to week 79 or something like that. So to make it more organized, we want the names of the current columns to become a variable called **week**, the values to become a variable called **rank** and we dropped the rows with missing values by using the **values_drop_na** argument.

```
fish_encounters1<- fish_encounters |>
  pivot_wider(names_from = station, values_from=seen, values_fill = 0)

fish_encounters1
```

A tibble: 19 x 12

	fish	Release	I80_1	Lisbon	Rstr	Base_TD	BCE	BCW	BCE2	BCW2	MAE	MAW
	<fct>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>
1	4842	1	1	1	1	1	1	1	1	1	1	1
2	4843	1	1	1	1	1	1	1	1	1	1	1
3	4844	1	1	1	1	1	1	1	1	1	1	1
4	4845	1	1	1	1	1	0	0	0	0	0	0
5	4847	1	1	1	0	0	0	0	0	0	0	0
6	4848	1	1	1	1	0	0	0	0	0	0	0
7	4849	1	1	0	0	0	0	0	0	0	0	0
8	4850	1	1	0	1	1	1	1	0	0	0	0

9	4851	1	1	0	0	0	0	0	0	0	0	0
10	4854	1	1	0	0	0	0	0	0	0	0	0
11	4855	1	1	1	1	1	0	0	0	0	0	0
12	4857	1	1	1	1	1	1	1	1	1	0	0
13	4858	1	1	1	1	1	1	1	1	1	1	1
14	4859	1	1	1	1	1	0	0	0	0	0	0
15	4861	1	1	1	1	1	1	1	1	1	1	1
16	4862	1	1	1	1	1	1	1	1	1	0	0
17	4863	1	1	0	0	0	0	0	0	0	0	0
18	4864	1	1	0	0	0	0	0	0	0	0	0
19	4865	1	1	1	0	0	0	0	0	0	0	0

- This dataset originally had one column called station, and in that column all of the different stations were named. So, we transformed it to a wider table, with a different column for each different station. The names of the columns come from the names of the stations and the values in seen are whether or not fish were detected at each station. Values_fill was changed from n/a to 0. It said n/a at first because it was only saying no record of fish was recorded but since that means there were 0 fish then we can replace the n/a with the value of 0.